CAP4730 – Assignment 4
Due April 6, 2026

OpenGL Viewer

In this assignment you will complete the viewer we started in the last homework.

- `Modelview Transformation`: Add a perspective projection matrix to your previous transformation. Show the effects of changing each parameter in the perspective matrix. `20pts`
- Implement functionality to show the contents of z-buffer in order to see the depth of points in the scene. Verify the depth values are consistent by rotating/scaling the scene. Examine the differences between z and z' coords. `20pts`
- Add lighting effects by implementing Gouraud and Phong shading. Show and describe the effects of each shading in your experiments. `20pts`
- Add texture mapping capability. Use or create a mesh that provides UV coordinates per vertex and extend your VBO (from the previous homework) to include these UV attributes. Load a texture (e.g., using `stb_image.h` or a procedurally generated pattern) and create an OpenGL texture object (handle `tex`) using `glGenTextures`, `glBindTexture`, and `glTexImage2D`, followed by appropriate filtering and wrapping parameters (and optionally `glGenerateMipmap`). In your vertex shader, pass UV coordinates to the fragment shader (`out vec2 UVcoord;`) and declare a corresponding input and a texture sampler in the fragment shader (`uniform sampler2D uTex;`). Sample the texture color at the fragment position using `vec3 texColor = texture(uTex, UVcoord).rgb;`. On the C++ side, after calling `glUseProgram(program)`, query the sampler location with `glGetUniformLocation(program, "uTex")` and set it to use texture unit 0 (a binding slot) via `glUniform1i(loc, 0)`. Finally, activate texture unit 0 with `glActiveTexture(GL_TEXTURE0)` and bind it to the texture object you created earlier with `glBindTexture(GL_TEXTURE_2D, tex)`. [1]. `20pts`
- Bonus: Provide a mechanism for flat shading and compare the results with Gouraud and Phong. `+10pts`
- Along with the source code and makefile (or a VS Code project file along with a README file for compiling instructions), submit a report (a PDF file) that describes and documents *each functionality* you implement. The report is graded on its comprehensiveness in documenting both the parts implemented and the parts that are partially (or not) implemented. `20pts`

This assignment can be done in groups of two students. You are welcome to discuss ideas/problems with other classmates but the source code and the report you submit MUST be your work. You need to clearly acknowledge sources (ideas, solutions, websites) that you use.

[1] see OGLDev Tutorial 16 for a reference.

Submission Guidelines
Submit to E-learning site a **single file** as a .zip or a .tar.gz bundle that contains all the files to be submitted. Include the source codes for your programs in the submission bundle. Please include a 'README' file that clearly explains how to run and test the program. Also include a 'Makefile' (or a VS Code project file) that compiles and links the program from the source files.
Late submissions are penalized by 20% of the grade for each day (up to 3) past the due date.